In this example, daily_returns, mean_return, and other variables inside the function are local—they exist only during the function's execution and can't be accessed from outside. This isolation is valuable in trading systems because it prevents one part of your program from accidentally modifying data used by another part.

Global Variables: Shared Across Functions

Variables defined outside any function are global—they can be accessed from anywhere in your program:

```
# Global variables - accessible from any function
risk_free_rate = 0.02
market_hours = ["9:30", "16:00"]

def calculate_excess_return(return_value):
    # This function can access the global risk_free_rate
    excess = return_value - risk_free_rate
    return excess

def is_market_open(current_time):
    # This function can access the global market_hours
    open_time = market_hours[0]
    close_time = market_hours[1]

    return open_time <= current_time < close_time

```

While global variables can be convenient for sharing constants (like risk_free_rate), they should be used sparingly in trading systems. Overuse of globals can lead to "spooky action at a distance," where one part of your program changes a value unexpectedly, affecting other parts.

Modifying Global Variables from Functions

If you need to modify a global variable from within a function, you must declare it using the global keyword:

```
# Global trading settings
position_size = 100
stop_loss_percent = 0.05
```